

Library Components Overview

This overview maps the major building blocks shipped by `humaine-explainerdashboard`, what each piece does, and the main technologies that power them. The goal is to give you a high-level map of the library rather than function-level detail.

Core Idea

- Wrap a trained, scikit-learn compatible model and its data in an `Explainer` that can compute SHAP values, permutation importances, partial dependence data, interaction effects, and task-specific metrics.
- Render those explanations as a Dash/Plotly web app (`ExplainerDashboard`) with reusable components and export options (live server or static HTML).
- Optionally host multiple dashboards together via `ExplainerHub`, or surface individual components inline inside notebooks.

Explainer Family

- `**BaseExplainer**`: Shared foundation that ingests models plus `X`/`y`, handles background data for SHAP, grouping of one-hot encoded categoricals, label/index metadata, cross-validated metrics, and persistence (`dump`, `from_file`, YAML config). It normalizes pipelines, manages positive-label handling for classification, and centralizes SHAP/permutation computations using `shap`, `pandas`, `numpy`, and `scikit-learn` metrics.
- `**ClassifierExplainer**`: Adds classification-specific behavior (ROC/PR curves, confusion matrices, threshold sweeps, lift/cumulative plots) with support for probability/log-odds outputs and multiclass positive-label selection.
- `**RegressionExplainer**`: Regression-focused summaries (residuals, preds-vs-actual, fit diagnostics) plus unit handling for targets. Uses the same SHAP/permutation machinery as the classifier variant.
- `**Model-specific shortcuts**`: Convenience subclasses for common estimators (e.g., random forest and XGBoost explainers) that preselect SHAP explainers and defaults for those model types.
- `**WeakspotAnalyzer**`: Slices the dataset (histogram- or tree-based) to flag regions where performance degrades relative to the global metric, for both classifiers and regressors. Supports metrics such as MSE/MAE/MAPE, accuracy, log-loss, and Brier score.

Dashboard Runtime

- `**ExplainerDashboard**`: The main Dash application wrapper. It assembles default or custom tabs (importances, model summary, individual predictions, what-if analysis, SHAP dependence/interaction, decision trees) using `plotly` visuals and `dash_bootstrap_components` layouts. Supports simple single-page mode, full multi-tab layouts, theming via Bootswatch CSS, tab/component toggles to control expensive computations, and download/export to static HTML. Runs with classic Dash or `jupyter_dash` servers (inline, external tab, or JupyterLab panes) and exposes a Flask server for production via `waitress`/`gunicorn`.
- `**Layout primitives**`: `ExplainerTabsLayout`/`ExplainerPageLayout` and the many `ExplainerComponent` primitives in `dashboard_components` provide the building blocks for custom dashboards. Composites cover common workflows (importances, model stats, contributions, what-if

editing, SHAP views, tree exploration) and can be mixed and matched or embedded in bespoke layouts.

- **InlineExplainer**: Notebook-friendly entry point that renders individual components or full tabs inline or in external windows using `JupyterDash`, making it easy to explore pieces of the dashboard without running a full app.

Plots and Explainability Components

- **Performance plots**: ROC/PR curves, lift and cumulative gain curves, confusion matrices, precision/recall/F1 vs cutoff, regression residuals and preds-vs-actual, all rendered with `plotly` and wired through Dash callbacks for live cutoff/label changes.

- **Importance and attribution**: SHAP summary, waterfall/contribution graphs, permutation importances, and grouped categorical importances. Uses `shap` backends for values, `plotly` for visual encoding, and dashboard toggles to sample data or switch sort/orientation.

- **Dependence and interaction**: Partial dependence plots (PDP), ICE-like overlays, SHAP dependence and interaction scatterplots with optional sampling and categorical grouping; relies on `plotly` for scatter/violin overlays and SHAP-derived interaction values when available.

- **Tree visualization**: Decision path tables and graphs plus individual tree diagrams via `dtreeviz` for supported tree ensembles (random forest, XGBoost, LightGBM, CatBoost when tree-based). Plots are embedded as images/interactive views in Dash.

- **What-if and editing**: Input editors let users tweak feature values, re-run predictions, and see updated contributions/PD plots inline; built on Dash form controls and explainer recomputation hooks.

- **Custom components**: Any `ExplainerComponent` can be subclassed or composed to create bespoke layouts, e.g., mixing a SHAP dependence view with a filtered contribution table. Components expose parameters to hide toggles, set defaults, and accept custom descriptions, making it straightforward to extend visuals while reusing explainer-backed data accessors.

ExplainerHub

- Hosts multiple `ExplainerDashboard` instances under one front page, with per-dashboard routes. Useful for model comparisons or exposing several production models together.

- Provides titles/descriptions per dashboard, optional iFrame sizing, and the ability to add/remove dashboards dynamically (including via URL patterns). Configuration can be saved/loaded from YAML alongside dumped explainers.

- Supports access control through `dash_auth` and `flask_simplelogin`, user management utilities, and optional partially open hubs (only some dashboards gated).

CLI & Configuration

- The `humaine-explainerdashboard` CLI (fork-specific name) exposes `run` and `build` commands via `click`. It can start dashboards from stored explainers (`.joblib`) or YAML dashboard configs, and can build explainers from model/data YAML definitions.

- Uses `waitress` as the default production-ready server, can auto-open browsers, and respects port overrides. CLI flows are suited for CI/CD where explainers/dashboards are regenerated as artifacts.

- YAML config helpers (``to_yaml``/``from_config``) persist dashboard layouts, explainer metadata, and user/login files for hubs.

Data, Assets, and Supporting Modules

- ****Datasets****: ``explainerdashboard.datasets`` ships sample data loaders (e.g., Titanic survival/fare datasets and helper metadata) to quickly demo dashboards.
- ****Plotting/metrics helpers****: ``explainer_plots.py`` plus methods on the explainer objects generate Plotly figures for SHAP summaries, PDPs, interaction plots, tree visualizations (``dtreeviz``), and performance charts.
- ****Static assets****: ``explainerdashboard/assets``, ``static``, and ``dashboard_components`` supply CSS/JS and reusable Dash layout pieces; dashboards can also export self-contained static HTML.

Model Coverage

- ****Estimator compatibility****: Built for any scikit-learn compatible estimator exposing ``fit``/``predict`` (and ``predict_proba`` for classifiers). Works with pipelines; transformers are applied before SHAP/permutation logic when possible.
- ****Tree ensembles****: First-class support for random forests, gradient boosting, XGBoost, LightGBM, and CatBoost; tree-based SHAP explainers are auto-selected and tree visualizations are enabled when the underlying library supports them.
- ****Linear and generalized linear models****: Logistic/linear regression and similar estimators leverage linear SHAP explainers and probability/log-odds outputs for classification dashboards.
- ****Neural nets via wrappers****: ``skorch`` and other sklearn-like wrappers can be used as long as they adhere to the estimator interface; SHAP falls back to kernel explainers when model-specific ones are unavailable.
- ****Custom models****: Any model with compatible predict interfaces can be wrapped; choose SHAP backend (``tree``, ``linear``, ``kernel``) explicitly when auto-guessing is insufficient, and provide background data when needed for probability-space outputs.

Technology Stack

- ****Modeling & math****: ``scikit-learn`` estimators and metrics, plus compatibility with ``xgboost``, ``catboost``, ``lightgbm``, and ``skorch`` wrappers; ``shap`` for explanations; ``numpy``/``pandas`` for data handling; ``dtreeviz`` for tree visualizations.
- ****Web UI****: ``dash`` + ``plotly`` for interactive charts, ``dash_bootstrap_components`` for layout and theming, ``JupyterDash`` for notebook serving, Flask under the hood, and ``waitress``/``gunicorn`` for deployment.
- ****Tooling & config****: ``click`` for the CLI, ``oyaml`` for stable YAML configs, ``joblib``/``pickle`` for persisting explainers and models.

Typical End-to-End Flow

- 1) Train a model (any scikit-learn compatible estimator) and wrap it in a ``ClassifierExplainer`` or ``RegressionExplainer``, optionally grouping categoricals and setting labels/units.
- 2) Launch an ``ExplainerDashboard`` (full or simple) to explore performance, feature importance, SHAP summaries, interactions, and what-if scenarios; export to HTML or YAML if needed.

3) Optionally combine multiple dashboards in an `ExplainerHub`, secure them with logins, and serve via the CLI using `waitress`.